Amazon Brazil

E-Commerce Data Analysis

Milestone 2

To identify trends, customer behaviours, and preferences that could be leveraged in the Indian market.

Vaishali Parmar

Table of Contents

Table of Contents 2
1. Introduction 3
2. Database Schema 4
3. Analysis I - Payment & Product Insights 5
4. Analysis II - Segmentation & Revenue 12
5. Analysis III - Advanced Analytics 17
6. Conclusion 24

1. Introduction

Amazon, a global leader in e-commerce, has achieved significant success across markets worldwide. In Brazil, Amazon connects small and medium businesses with millions of customers, establishing itself as a key marketplace player.

Given the similarities between Brazil and India - large populations, diverse consumer bases, and growing digital economies - there is a strategic opportunity to apply insights from the Brazilian market to inform Amazon India's growth strategy.

This report presents a comprehensive SQL-based analysis of Amazon Brazil's e-commerce data, organized into three progressive analysis phases covering payment patterns, customer segmentation, product performance, and sales trends.

2. Database Schema

The analysis is built on six interconnected tables within the amazon_brazil schema in PostgreSQL:

Table	Description
`customers`	Customer demographics and location data (customer_id, unique_id, zip_code)
`orders`	Order lifecycle tracking with timestamps (purchase, approval, delivery)
`order_items`	Line-item details including price, freight, and seller information
`product`	Product specifications - category, dimensions, weight, and photos
`seller`	Seller profiles with location identifiers
`payments`	Transaction records - payment type, installments, and values

3. Analysis I - Payment & Product Insights

This section examines payment distributions, product pricing patterns, and data quality across the Amazon Brazil marketplace.

Q1:

Problem Statement: Round the average payment values to integer for each payment type and display results sorted in ascending order.

Analysis Approach: Used AVG() with ROUND() on the payments table, filtering out undefined payment types, grouped by payment_type and ordered ascending by the rounded average.

SQL Query:

```
SELECT payment_type,
  ROUND(AVG(payment_value), 0) AS rounded_avg_payment
FROM amazon_brazil.payments
WHERE payment_type <> 'not_defined'
GROUP BY payment_type
ORDER BY rounded_avg_payment ASC;
```

Output:

	payment_type character varying (30)	rounded_avg_payment numeric
1	voucher	66
2	debit_card	143
3	boleto	145
4	credit_card	163

Recommendation: Credit cards show the highest average payment value. Amazon India should prioritize credit card partnerships and offer EMI options to drive higher-value transactions.

Q2:

Problem Statement: Calculate the percentage of total orders for each payment type, rounded to one decimal place, and display in descending order.

Analysis Approach: Used COUNT(*) divided by a subquery for total payments count, multiplied by 100 and rounded to 1 decimal. Grouped by payment_type, ordered descending.

SQL Query:

```
SELECT payment_type,
  ROUND(COUNT(*) * 100.0 /
    (SELECT COUNT(*) FROM amazon_brazil.payments), 1)
  AS percentage_orders
FROM amazon_brazil.payments
GROUP BY payment_type
ORDER BY percentage_orders DESC;
```

Output:

	payment_type character varying (30)	percentage_orders numeric
1	credit_card	73.9
2	boleto	19.0
3	voucher	5.6
4	debit_card	1.5
5	not_defined	0.0

Recommendation: Credit cards dominate order share. Amazon India should ensure seamless credit card checkout while also promoting UPI and digital wallets to diversify payment options.

Q3:

Problem Statement: Identify all products priced between 100 and 500 BRL that contain the word 'Smart' in their category name, sorted by price descending.

Analysis Approach: Joined the product and order_items tables on product_id. Applied BETWEEN for price range and ILIKE for case-insensitive pattern matching on category name. Used DISTINCT to avoid duplicates.

SQL Query:

```
SELECT DISTINCT p.product_id, oi.price
FROM amazon_brazil.product p
JOIN amazon_brazil.order_items oi
  ON p.product_id = oi.product_id
WHERE oi.price BETWEEN 100 AND 500
  AND p.product_category_name ILIKE '%smart%'
ORDER BY oi.price DESC;
```

Output:

	product_id character varying (50)	price numeric (10,2)
1	1df1a2df8ad2b9d3aa49fd851e314...	439.99
2	7debe59b10825e89c1cbcc8b190c...	349.99
3	ca86b9fe16e12de698c955aedff0ae...	349.00
4	0e52955ca8143bd179b311cc454a...	335.00
5	7aeaa8f3e592e380c420e8910a717...	329.90
6	d1b571cd58267d8cac8b2afd6e288...	299.90
7	66ffe28d0fd53808d0535eee4b90a...	254.00
8	f06796447de379a26dde5fcac6a1a...	239.90
9	d3d5a1d52abe9a7d234908d873fc3...	229.90
10	06ae026e430189633c2fbd0288c86...	217.36
11	49ef750dc5bf23e3788d4f614bc6d...	198.00
12	33bb7da523efcdef6cd2996cbf72d...	148.00
13	6f5795735ab2c629b22669fe889b7...	129.90
14	3626035966a7aaee90d68108caeb...	124.90
15	aeaba104830f91586dae1bff90f54a...	123.90
16	630c84b1ce83ae0e9ddc05a14103...	110.00

Total rows: 19 Query complete 00:00:00.260

Recommendation: Smart products in the 100–500 BRL range represent a growing mid-tier segment. Amazon India should curate a dedicated smart devices storefront with bundled offers.

Q4:

Problem Statement: Determine the top 3 months with the highest total sales value, rounded to the nearest integer.

Analysis Approach: Joined orders with order_items. Extracted month from order_purchase_timestamp using EXTRACT(). Aggregated with SUM and ROUND, ordered descending, limited to 3.

SQL Query:

```
SELECT
  EXTRACT(MONTH FROM o.order_purchase_timestamp) AS month,
  ROUND(SUM(oi.price)) AS total_sales
FROM amazon_brazil.orders o
JOIN amazon_brazil.order_items oi
  ON o.order_id = oi.order_id
GROUP BY month
ORDER BY total_sales DESC
LIMIT 3;
```

Output:

	month numeric	total_sales numeric
1	5	1502589
2	8	1428658
3	7	1393539

Recommendation: Peak sales months likely coincide with festival or holiday periods. Amazon India should align major sale events (like Great Indian Festival) with these high-performing months.

Q5:

Problem Statement: Find product categories where the difference between maximum and minimum product prices exceeds 500 BRL.

Analysis Approach: Joined product and order_items tables. Used MAX() and MIN() on price, grouped by category. Applied HAVING clause to filter categories with price difference > 500.

SQL Query:

```
SELECT p.product_category_name,
  ROUND(MAX(oi.price) - MIN(oi.price)) AS price_difference
FROM amazon_brazil.product p
JOIN amazon_brazil.order_items oi
  ON p.product_id = oi.product_id
GROUP BY p.product_category_name
HAVING (MAX(oi.price) - MIN(oi.price)) > 500
ORDER BY price_difference DESC;
```

Output:

	product_category_name character varying (50)	price_difference numeric
1	utilidades_domesticas	6732
2	pcs	6695
3	artes	6496
4	eletroportateis	4793
5	instrumentos_musicais	4395
6	consoles_games	4095

Total rows: 57 Query complete 00:00:00.155

Recommendation: Categories with large price spreads indicate diverse product tiers. Amazon India can introduce tiered pricing labels (Budget, Standard, Premium) to help customers navigate these categories.

Q6:

Problem Statement: Identify payment types with the most consistent transaction amounts by finding those with the smallest standard deviation.

Analysis Approach: Calculated STDDEV() of payment_value per payment_type, cast to NUMERIC for rounding to 2 decimal places. Ordered ascending to show most consistent first.

SQL Query:

```
SELECT payment_type,
  ROUND(STDDEV(payment_value)::NUMERIC, 2) AS std_deviation
FROM amazon_brazil.payments
GROUP BY payment_type
ORDER BY std_deviation ASC;
```

Output:

	payment_type character varying (30)	std_deviation numeric
1	not_defined	0.00
2	voucher	115.52
3	boleto	213.58
4	credit_card	222.12
5	debit_card	245.79

Recommendation: Payment types with low variance (like debit cards) indicate predictable transaction behaviour. These are ideal for subscription-based services or auto-pay features.

Q7:

Problem Statement: Retrieve products where the product category name is missing or contains only a single character.

Analysis Approach: Filtered the product table using IS NULL for missing names and LENGTH() = 1 for single-character entries. Ordered by product_id for systematic review.

SQL Query:

```
SELECT product_id, product_category_name
FROM amazon_brazil.product
WHERE product_category_name IS NULL
  OR LENGTH(product_category_name) = 1
ORDER BY product_id;
```

Output:

	product_id [PK] character varying (50)	product_category_name character varying (50)
1	0082684bb4a60a862baaf7a60a58...	[null]
2	00ab8a8b9fe219511dc3f178c6d79...	[null]
3	00d62b338366db4c4aec8547ea8f...	[null]
4	0103863bf3441460142ec23c7438...	[null]
5	014fcf6bd5cd4c7ee29fb3bb618c4...	[null]
6	017457b0013d01d5a5a4a020ed1f...	[null]

Total rows: 614 Query complete 00:00:00.101

Recommendation: Data quality issues in product categorization can impact search and recommendation systems. Amazon India should implement mandatory category validation during product listing.

4. Analysis II - Segmentation & Revenue

This section explores order value segmentation, customer classification, and category-level revenue analysis to inform strategic decisions.

Q1:

Problem Statement: Segment order values into Low (<200), Medium (200–1000), and High (>1000) ranges. Count each payment type within these segments.

Analysis Approach: Used CASE WHEN to classify payment_value into three segments. Grouped by both segment and payment_type, counted occurrences, and ordered by count descending.

SQL Query:

```
SELECT
  CASE
    WHEN payment_value < 200 THEN 'low(<200)'
    WHEN payment_value BETWEEN 200 AND 1000
      THEN 'medium(200-1000)'
    WHEN payment_value > 1000 THEN 'high(>1000)'
  END AS order_value_segment,
  payment_type,
  COUNT(*) AS count
FROM amazon_brazil.payments
GROUP BY order_value_segment, payment_type
ORDER BY count DESC;
```

Output:

	order_value_segment text	payment_type character varying (30)	count bigint
1	low(<200)	credit_card	60548
2	low(<200)	boleto	16444
3	medium(200-1000)	credit_card	15303
4	low(<200)	voucher	5476
5	medium(200-1000)	boleto	3162
6	low(<200)	debit_card	1287

Total rows: 13 Query complete 00:00:00.105

Recommendation: Low-value orders dominate across all payment types. Amazon India should introduce micro-payment solutions and encourage upselling strategies to shift orders toward the medium segment.

Q2:

Problem Statement: Calculate the minimum, maximum, and average price for each product category, sorted by average price descending.

Analysis Approach: Joined product and order_items tables. Applied MIN(), MAX(), AVG() aggregate functions on price, all rounded to integers. Grouped by category, ordered by avg_price descending.

SQL Query:

```
SELECT p.product_category_name,
  ROUND(MIN(oi.price)) AS min_price,
  ROUND(MAX(oi.price)) AS max_price,
  ROUND(AVG(oi.price)) AS avg_price
FROM amazon_brazil.product p
JOIN amazon_brazil.order_items oi
  ON p.product_id = oi.product_id
GROUP BY p.product_category_name
ORDER BY avg_price DESC;
```

Output:

	product_category_name character varying (50)	min_price numeric	max_price numeric	avg_price numeric
1	pcs	35	6729	1098
2	portateis_casa_forno_e_cafe	10	2899	624
3	eletrodomesticos_2	14	2350	476
4	agro_industria_e_comercio	13	2990	342
5	instrumentos_musicais	5	4400	282
6	eletroportateis	7	4799	281

Total rows: 79 Query complete 00:00:00.167

Recommendation: High average-price categories represent premium segments. Amazon India should focus on quality assurance and premium delivery options for these categories to maintain customer trust.

Q3:

Problem Statement: Find all customers with more than one order and display their unique IDs along with total number of orders placed.

Analysis Approach: Joined customers and orders tables on customer_id. Grouped by customer_unique_id, used HAVING COUNT > 1 to filter repeat buyers, and ordered by total_orders descending.

SQL Query:

```
SELECT
  c.customer_unique_id,
  COUNT(o.order_id) AS total_orders
FROM amazon_brazil.customers c
JOIN amazon_brazil.orders o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_unique_id
HAVING COUNT(o.order_id) > 1
ORDER BY total_orders DESC;
```

Output:

	customer_unique_id character varying (50)	total_orders bigint
1	a91e80fbe80ddc07de66a5cf927029...	16
2	a6168cd79131e64acef92e3c74d6cc...	16
3	363f980585bf04c1a88fdb986011c5...	16
4	cbd0350d4ccba9772e8e768d4a4a5...	16
5	417b909c0962b2610f1cfeb1c14789...	16
6	5f94af52aef02c968a2e0f01f430864e	16

Total rows: 3140 Query complete 00:00:00.987

Recommendation: Repeat customers are the backbone of e-commerce revenue. Amazon India should implement personalized re-engagement campaigns and early access to deals for multi-order customers.

Q4:

Problem Statement: Label customers as New (1 order), Returning between2 and 4 orders, or Loyal (>4 orders) based on purchase history using a temporary table.

Analysis Approach: Created a temporary table aggregating order counts per customer. Then applied CASE WHEN logic to classify each customer into New, Returning, or Loyal segments.

SQL Query:

```
CREATE TEMP TABLE customer_order_counts AS
SELECT c.customer_unique_id,
  COUNT(o.order_id) AS total_orders
FROM amazon_brazil.customers c
JOIN amazon_brazil.orders o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_unique_id;

SELECT customer_unique_id,
  CASE
    WHEN total_orders = 1 THEN 'New'
    WHEN total_orders BETWEEN 2 AND 4 THEN 'Returning'
    WHEN total_orders > 4 THEN 'Loyal'
  END AS customer_type
FROM customer_order_counts
ORDER BY total_orders DESC;
```

Output:

	customer_unique_id character varying (50)	customer_type text
1	6af40347f5dd7bdd65437a35e1b2f...	Loyal
2	7f4f709af2fd8fea44aacd30bca462...	Loyal
3	417b909c0962b2610f1cfeb1c1478...	Loyal
4	75f15790b1852b42b1dbf645d98ff...	Loyal
5	0fdc0d21e1983e8af4d399e17671f...	Loyal
6	363f980585bf04c1a88fdb986011c...	Loyal

Total rows: 95077 | Query complete 00:00:00.202

Recommendation: The majority of customers are likely 'New'. Amazon India should design onboarding journeys with first-order discounts and follow-up recommendations to convert New into Returning customers.

Q5:

Problem Statement: Identify the top 5 product categories that generate the most revenue.

Analysis Approach: Joined product and order_items tables. Aggregated total revenue per category using SUM(price) with ROUND. Ordered by total_revenue and limited to 5 results.

SQL Query:

```
SELECT p.product_category_name,
  ROUND(SUM(oi.price)) AS total_revenue
FROM amazon_brazil.product p
JOIN amazon_brazil.order_items oi
  ON p.product_id = oi.product_id
GROUP BY product_category_name
ORDER BY total_revenue DESC
LIMIT 5;
```

Output:

	product_category_name character varying (50)	total_revenue numeric
1	f	22
2	w	60
3	seguros_e_servicos	283
4	t	413
5	fashion_roupa_infanto_juve...	570

Recommendation: Top revenue categories should receive priority in inventory management, marketing spend, and seller onboarding. Amazon India should ensure adequate stock levels and competitive pricing in these categories.

5. Analysis III - Advanced Analytics

This section exmines subqueries, CTEs, window functions, and cumulative aggregations for deeper analytical insights into seasonal trends, customer loyalty, and growth patterns.

Q1:

Problem Statement: Calculate total sales for each season (Spring: Mar to May, Summer: Jun to Aug, Autumn: Sep to Nov, Winter: Dec to Feb).

Analysis Approach: Used a subquery with CASE WHEN on EXTRACT(MONTH) to assign seasons. The outer query then aggregated total sales per season using SUM with ROUND, ordered descending.

SQL Query:

```
SELECT season,
  ROUND(SUM(price)) AS total_sales
FROM (
  SELECT
    CASE
      WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp)
        IN (3,4,5) THEN 'spring'
      WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp)
        IN (6,7,8) THEN 'summer'
      WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp)
        IN (9,10,11) THEN 'autumn'
      WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp)
        IN (1,2,12) THEN 'winter'
    END AS season,
    oi.price
  FROM amazon_brazil.orders o
  JOIN amazon_brazil.order_items oi
    ON oi.order_id = o.order_id
) AS seasonal_data
GROUP BY season
ORDER BY total_sales DESC;
```

Output:

	season text	total_sales numeric
1	spring	4216722
2	summer	4120360
3	winter	2905750
4	autumn	2348813

Recommendation: Seasonal sales trends should guide Amazon India's marketing calendar. Allocate higher ad budgets and flash sales during peak seasons, and run clearance events during slower ones.

Q2:

Problem Statement: Find products whose total quantity sold is above the overall average quantity sold per product.

Analysis Approach: Used a HAVING clause with a correlated subquery. The subquery calculates the average of per-product totals. The main query filters products exceeding this threshold.

SQL Query:

```
SELECT product_id,
  COUNT(*) AS total_quantity_sold
FROM amazon_brazil.order_items
GROUP BY product_id
HAVING COUNT(*) > (
  SELECT AVG(total_qty)
  FROM (
    SELECT product_id, COUNT(*) AS total_qty
    FROM amazon_brazil.order_items
    GROUP BY product_id
  ) AS qty_per_product
)
ORDER BY total_quantity_sold DESC;
```

Output:

	product_id character varying (50)	total_quantity_sold bigint
1	aca2eb7d00ea1a7b8ebd4e6831466...	527
2	99a4788cb24856965c36a24e339b6...	488
3	422879e10f46682990de24d770e7f8...	484
4	389d119b48cf3043d311335e499d9...	392
5	368c6c730842d78016ad823897a37...	388
6	53759a2ecddad2bb87a079a1f1519f...	373

Total rows: 6366 Query complete 00:00:00.289

Recommendation: Above-average sellers are proven demand drivers. Amazon India should ensure these products have Prime eligibility, featured placements, and robust inventory to prevent stockouts.

Q3:

Problem Statement: Calculate total revenue generated each month in 2018 to identify periods of peak and low sales.

Analysis Approach: Joined orders and order_items, filtered for year 2018 using EXTRACT(YEAR). Grouped by month with SUM(price), ordered by total_revenue ascending for trend visualization.

SQL Query:

```
SELECT
  EXTRACT(MONTH FROM order_purchase_timestamp) AS month,
  ROUND(SUM(oi.price)) AS total_revenue
FROM amazon_brazil.orders o
JOIN amazon_brazil.order_items oi
  ON o.order_id = oi.order_id
WHERE EXTRACT(YEAR FROM order_purchase_timestamp) = 2018
GROUP BY month
ORDER BY total_revenue ASC;
```

Output:

	month numeric	total_revenue numeric
1	9	145
2	2	844179
3	8	854686
4	6	865124
5	7	895507
6	1	950030

Total rows: 9 | Query complete 00:00:00.224

Recommendation: Monthly revenue trends reveal cyclical patterns. Amazon India should use this data for demand forecasting, warehouse staffing, and logistics planning throughout the year.

Q4:

Problem Statement: Segment customers by purchase frequency: Occasional (1,2 orders), Regular (3,4,5), Loyal (5+). Count each segment.

Analysis Approach: Used a CTE (WITH clause) to classify customers by order count using CASE WHEN. The outer query then counted customers in each segment, ordered by count descending.

SQL Query:

```
WITH orders_counts AS (
  SELECT c.customer_unique_id,
    COUNT(o.order_id) AS total_orders,
    CASE
      WHEN COUNT(o.order_id) <= 2 THEN 'occasional'
      WHEN COUNT(o.order_id) BETWEEN 3 AND 5 THEN 'regular'
      WHEN COUNT(o.order_id) >= 5 THEN 'loyal'
    END AS customer_type
  FROM amazon_brazil.customers c
  JOIN amazon_brazil.orders o
    ON c.customer_id = o.customer_id
  GROUP BY customer_unique_id
)
SELECT customer_type, COUNT(*) AS count
FROM orders_counts
GROUP BY customer_type
ORDER BY count DESC;
```

Output:

	customer_type text	count bigint
1	occational	94635
2	regular	333
3	loyal	109

Recommendation: The Occasional segment is typically the largest. Amazon India should design targeted retention programs - loyalty points, exclusive deals, and personalized recommendations - to move customers up the loyalty.

Q5:

Problem Statement: Rank customers based on their average order value to find the top 20 high-value customers.

Analysis Approach: Used a CTE to calculate average order value per customer. Applied RANK() window function ordered by avg_order_value descending. Limited output to top 20.

SQL Query:

```
WITH customer_avg AS (
  SELECT o.customer_id,
    ROUND(AVG(oi.price)) AS avg_order_value
  FROM amazon_brazil.orders o
  JOIN amazon_brazil.order_items oi
    ON o.order_id = oi.order_id
  GROUP BY customer_id
)
SELECT customer_id, avg_order_value,
  RANK() OVER (ORDER BY avg_order_value DESC)
    AS customer_rank
FROM customer_avg
ORDER BY customer_rank
LIMIT 20;
```

Output:

	customer_id character varying (50)	avg_order_value numeric	customer_rank bigint
1	c6e2731c5b391845f6800c97401a4...	6735	1
2	f48d464a0baaea338cb25f816991ab...	6729	2
3	3fd6777bbce08a352fddd04e4a7cc8...	6499	3
4	df55c14d1476a9a3467f131269c24...	4799	4
5	24bbf5fd2f2e1b359ee7de94defc4a15	4690	5
6	3d979689f636322c62418b6346b1c...	4590	6

Total rows: 20 | Query complete 00:00:01.006

Recommendation: Top 20 high-value customers are ideal candidates for an exclusive rewards program. Amazon India should offer them priority support, early access to new products, and premium membership benefits.

Q6:

Problem Statement: Calculate the running (cumulative) total of sales for each product month by month from its first sale.

Analysis Approach: Used a CTE to compute monthly sales per product via DATE_TRUNC and SUM. Applied SUM() as a window function with PARTITION BY product_id and ORDER BY sale_month for cumulative totals.

SQL Query:

```
WITH monthly_sales AS (
  SELECT oi.product_id,
    DATE_TRUNC('month', o.order_purchase_timestamp)
      AS sale_month,
    SUM(oi.price) AS monthly_sales
  FROM amazon_brazil.orders o
  JOIN amazon_brazil.order_items oi
    ON o.order_id = oi.order_id
  GROUP BY oi.product_id,
    DATE_TRUNC('month', o.order_purchase_timestamp)
)
SELECT product_id, sale_month,
  ROUND(SUM(monthly_sales) OVER (
    PARTITION BY product_id
    ORDER BY sale_month
  )) AS total_sales
FROM monthly_sales
ORDER BY product_id, sale_month;
```

Output:

	product_id character varying (50)	sale_month timestamp without time zone	total_sales numeric
1	00066f42aeeb9f3007548bb9d3f33c38	2018-05-01 00:00:00	102
2	00088930e925c41fd95ebfe695fd2655	2017-12-01 00:00:00	130
3	0009406fd7479715e4bef61dd91f2462	2017-12-01 00:00:00	229
4	000b8f95fcb9e0096488278317764d...	2018-08-01 00:00:00	118
5	000d9be29b5207b54e86aa1b1ac54...	2018-04-01 00:00:00	199
6	0011c512eb256aa0dbbb544d8dffcf6e	2017-12-01 00:00:00	52

Total rows: 62176 Query complete 00:00:01.720

Recommendation: Cumulative sales trends help identify product lifecycle stages. Amazon India should use this data to spot declining products early and plan promotional boosts or phase-out strategies.

Q7:

Problem Statement: Compute total monthly sales per payment type in 2018 and calculate month-over-month growth rate.

Analysis Approach: Used a CTE to aggregate monthly totals per payment type for 2018. Applied LAG() window function to access previous month's value and calculated percentage change.

SQL Query:

```
WITH total_sale AS (
  SELECT p.payment_type,
    EXTRACT(MONTH FROM o.order_purchase_timestamp)
      AS sale_month,
    ROUND(SUM(oi.price)) AS monthly_total
  FROM amazon_brazil.orders o
  JOIN amazon_brazil.order_items oi
    ON o.order_id = oi.order_id
  JOIN amazon_brazil.payments p
    ON o.order_id = p.order_id
  WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018
  GROUP BY p.payment_type, sale_month
)
SELECT payment_type, sale_month, monthly_total,
  CASE
    WHEN LAG(monthly_total) OVER () = 0 THEN NULL
    ELSE ROUND(
      (monthly_total - LAG(monthly_total) OVER ())
      / LAG(monthly_total) OVER () * 100
    )
  END AS monthly_change
FROM total_sale;
```

Output:

	payment_type character varying (30)	sale_month numeric	monthly_total numeric	monthly_change numeric
1	boleto	1	170651	[null]
2	boleto	2	153166	-10
3	boleto	3	157807	3
4	boleto	4	162941	3
5	boleto	5	166572	2
6	boleto	6	126380	-24

Total rows: 33 Query complete 00:00:00.480

Recommendation: Month-over-month growth analysis by payment type helps identify emerging trends. Amazon India should invest in payment methods showing consistent growth and address declining ones with targeted promotions.

6. Conclusion

This analysis of Amazon Brazil's e-commerce data reveals several actionable insights for Amazon India's market strategy. Credit cards dominate payment preferences, seasonal patterns significantly influence revenue, and the vast majority of customers remain single-order buyers - representing a major retention opportunity.

Key takeaways include the need for diversified payment options (particularly digital wallets and UPI for India), targeted loyalty programs to convert one-time buyers into repeat customers, and data-driven inventory planning based on seasonal and monthly sales trends.

By applying these cross-market learnings, Amazon India can enhance customer experience, optimize revenue, and strengthen its competitive position in the rapidly growing Indian e-commerce landscape.